

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – DEBUGGING & OPTIMISATION TASK

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	Assessment:	Assessment Tool 3 – Debugging & Optimisation Task
Weightage:	10% of CIA	Total Marks:	100
CO2 Statement:	<i>Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)</i>		
Student Name:	K ABISHEK	Reg. No.:	192521120
Section / Batch:	IT	Date:	30/7/26

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given program/code segment before identifying errors.
- Clearly identify logical, syntactical, and performance-related issues in the code.
- Apply systematic debugging techniques to correct the errors effectively.
- Optimize the given solution by improving efficiency, reducing unnecessary computations, or enhancing time complexity wherever possible.
- Provide proper justification for all corrections and optimization decisions.
- Write clean, structured, and well-documented code with appropriate comments.
- Maintain clarity, logical reasoning, and technical accuracy throughout the solution.

Question Type	Focus Area	Questions
Debugging & Optimisation Task	Identify errors, debug programs, and optimize algorithm efficiency using appropriate techniques	Q1

SECTION A – DEBUGGING & OPTIMISATION TASK

Code of Conduct

I K ABISHEK/ 192521120 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: K ABISHEK

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%				
Debugging	25%				
Optimization	20%				
Analysis	20%				
Code Quality	15%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name: _____	Name: _____	Name: _____
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

ASSESSMENT 3 – DEBUGGING AND OPTIMIZATION TASK

Debugging String Matching Code – Skipped Last Valid Position

Problem Statement

A brute force string matching algorithm is used to find all occurrences of a pattern within a given text. The provided implementation contains an error in the loop boundary, causing it to skip the last valid starting position of the pattern.

The objective is to inspect the given code, identify the bug, explain why the last possible alignment is not checked, correct the implementation, and optimize the code for clarity while preserving the brute force string matching behavior.

Given Code

```
def string_match(text, pattern):  
    n, m = len(text), len(pattern)  
    matches = []  
  
    for i in range(n - m):  
        for j in range(m):  
            if text[i+j] != pattern[j]:  
                break  
        if j == m-1:  
            matches.append(i)  
  
    return matches
```

1. Understanding the Problem

The brute force string matching algorithm compares the pattern with the text at every possible starting position.

For a text of length **n** and a pattern of length **m**, the pattern can start at positions:

0, 1, 2, ..., n - m

Therefore, the total number of valid starting positions is:

n - m + 1

The given program uses:

```
for i in range(n - m):
```

However, Python's range() excludes its upper limit.

Therefore, the loop checks only:

0, 1, 2, ..., n - m - 1

and completely skips:

i = n - m

This is the last valid starting position of the pattern.

2. Identifying the Bug

The incorrect statement is:

```
for i in range(n - m):
```

The loop should be:

```
for i in range(n - m + 1):
```

The reason is that the last valid alignment occurs when:

i + m = n

Therefore:

i = n - m

Since this position must also be checked, the range must include n - m.

Example

Consider:

```
Text = "ABCDE"
```

```
Pattern = "DE"
```

Here:

n = 5

m = 2

The valid starting positions are:

0, 1, 2, 3

because:

$$n - m = 5 - 2 = 3$$

The original loop:

```
range(5 - 2)
```

becomes:

```
range(3)
```

which generates:

0, 1, 2

Therefore, position **3** is skipped even though "DE" starts at index 3.

The corrected loop:

```
range(5 - 2 + 1)
```

becomes:

```
range(4)
```

which generates:

0, 1, 2, 3

Thus, every valid alignment is examined.

3. Debugging the Code

Original Code

```
for i in range(n - m):
```

Corrected Code

```
for i in range(n - m + 1):
```

This single change ensures that the algorithm examines every possible starting position.

4. Corrected Brute Force Implementation

```
def string_match(text, pattern):
```

```
    n = len(text)
```

```
    m = len(pattern)
```

```
    matches = []
```

```

# Check every valid starting position
for i in range(n - m + 1):

    # Compare the pattern with the text
    for j in range(m):
        if text[i + j] != pattern[j]:
            break
    else:
        # Pattern matched completely
        matches.append(i)

return matches

```

The corrected implementation preserves the original brute force approach. It does not use advanced string-matching algorithms such as KMP or Rabin-Karp.

5. Testing the Corrected Program

Test Case 1

Text = "ABCDE"

Pattern = "DE"

Expected result:

[3]

Execution:

Starting position 0 → "AB" → No match

Starting position 1 → "BC" → No match

Starting position 2 → "CD" → No match

Starting position 3 → "DE" → Match

Output:

[3]

The last valid position is now successfully checked.

6. Test Case 2 – Multiple Matches

Consider:

Text = "ABABA"

Pattern = "ABA"

Here:

n = 5

m = 3

Valid starting positions:

0, 1, 2

The pattern occurs at:

Position 0 → ABA

Position 2 → ABA

Output:

[0, 2]

The corrected program successfully identifies both occurrences.

7. Test Case 3 – Pattern at the Last Position

Consider:

Text = "HELLOWORLD"

Pattern = "WORLD"

The pattern starts at index:

5

Here:

n = 10

m = 5

Therefore:

n - m = 5

The original loop:

range(5)

checks positions:

0, 1, 2, 3, 4

and skips position **5**.

The corrected loop:

range(6)

checks:

0, 1, 2, 3, 4, 5

Therefore, the corrected program finds:

[5]

8. Improved Implementation with Proper Comments

The following version improves the clarity of the original implementation while preserving the brute force behavior.

```
def string_match(text, pattern):
```

```
    """
```

```
    Find all occurrences of pattern in text
```

```
    using the brute force string matching method.
```

```
    """
```

```
    n = len(text)
```

```
    m = len(pattern)
```

```
    matches = []
```

```
    # If the pattern is empty, return no match positions.
```

```
    if m == 0:
```



```
return matches
```

```
# Check every possible starting position of the pattern.
```

```
for i in range(n - m + 1):
```

```
    # Assume the pattern matches at this position.
```

```
    match = True
```

```
    # Compare each character of the pattern.
```

```
    for j in range(m):
```

```
        if text[i + j] != pattern[j]:
```

```
            match = False
```

```
            break
```

```
    # Store the starting position if the pattern matched.
```

```
    if match:
```

```
        matches.append(i)
```

```
return matches
```

```
# Example input
```

```
text = "ABABA"
```

```
pattern = "ABA"
```

```
# Find all pattern occurrences
```

```
result = string_match(text, pattern)
```

```
print("Text:", text)
print("Pattern:", pattern)
print("Matching positions:", result)
```

9. Sample Output

Text: ABABA

Pattern: ABA

Matching positions: [0, 2]

The output shows that the pattern "ABA" occurs at starting positions **0** and **2**.

10. Debugging Verification

The corrected program can be verified using a pattern that occurs exactly at the last possible position.

Input

Text = "ABCDE"

Pattern = "DE"

Valid Starting Positions

0, 1, 2, 3

Comparison Process

Position 0 → AB ≠ DE

Position 1 → BC ≠ DE

Position 2 → CD ≠ DE

Position 3 → DE = DE

Output

[3]

This confirms that the last valid position is no longer skipped.

11. Comparison Between Original and Corrected Code

Feature	Original Code	Corrected Code
Starting positions	$\text{range}(n-m)$	$\text{range}(n-m+1)$
Last valid position checked	No	Yes
All alignments checked	No	Yes
Brute force behavior	Yes	Yes
Correct for pattern at end	No	Yes
Readability	Basic	Improved
Correct result	Not always	Yes

The main correction is the addition of **+1** to the outer loop boundary.

12. Time Complexity Analysis

Let:

n = length of the text

m = length of the pattern

The algorithm checks at most:

$n - m + 1$

possible starting positions.

At each position, up to **m** characters may be compared.

Therefore, the maximum number of character comparisons is approximately:

$(n - m + 1) \times m$

Hence, the worst-case time complexity is:

$O((n - m + 1)m)$

which is commonly expressed as:

$O(nm)$

Best Case

If a mismatch occurs at the first character of each alignment, only one comparison may be needed at each position.

Therefore:

Best Case = $O(n)$

Worst Case

If the pattern and text contain many matching characters before a mismatch occurs, almost all m characters may be compared at every starting position.

Therefore:

Worst Case = $O(nm)$

Space Complexity

The algorithm stores only the list of matching positions and a few variables.

Ignoring the output list, the auxiliary space is:

$O(1)$

If the output list containing all matches is included, the total space can be:

$O(k)$

where k is the number of matches found.

13. Why the Correction Is Necessary

The pattern can legally start at position $n - m$.

For example:

Text length = 10

Pattern length = 5

The last possible starting position is:

$$10 - 5 = 5$$

The pattern occupies:

5, 6, 7, 8, 9

which is completely inside the text.

Therefore, position 5 must be examined.

The original:

`range(n - m)`

does not include position $n - m$.

The corrected:

`range(n - m + 1)`

does include it.

Therefore, the corrected loop checks every valid alignment.

14. Optimization for Clarity

The corrected implementation improves the original code in several ways:

- The loop boundary explicitly includes every valid alignment.
- Variable names such as n , m , i , and j are used consistently.
- The match variable clearly indicates whether the pattern matched.
- Comments explain the purpose of each major operation.
- The implementation remains a brute force algorithm.
- No advanced string-matching technique is introduced.
- The code is easier to read, test, and maintain.

The optimization is therefore focused on **clarity and correctness**, rather than changing the underlying algorithm.

15. Correctness Verification

The corrected algorithm examines every starting position from:

0 to $n - m$

For each position, it compares the pattern characters with the corresponding text characters.

If every character matches, the starting position is added to the result list.

Therefore:

1. Every valid alignment is checked.
2. No valid starting position is skipped.
3. A position is added only when the complete pattern matches.
4. Multiple occurrences of the pattern can be identified.

5. The last possible occurrence is correctly detected.

Hence, the corrected implementation produces the correct set of matching positions.

16. Final Result

Original Problem

The original implementation skips the last valid starting position because:

`range(n - m)`

does not include $n - m$.

Correction

`range(n - m + 1)`

Corrected Example

Text = "HELLOWORLD"

Pattern = "WORLD"

Result

[5]

The pattern is successfully detected at the final valid starting position.

Complexity

Best Case Time Complexity = $O(n)$

Worst Case Time Complexity = $O(nm)$

Auxiliary Space Complexity = $O(1)$

17. Conclusion

The debugging process identified an off-by-one error in the outer loop of the brute force string matching implementation. The original loop used `range(n - m)`, which excluded the last valid starting position $n - m$.

Changing the loop to `range(n - m + 1)` ensures that every valid alignment between the pattern and text is examined.

The improved implementation preserves the brute force string matching approach while making the code clearer, easier to understand, and correct for patterns occurring at any valid position, including the final position.

The corrected algorithm has $O(nm)$ worst-case time complexity and $O(1)$ auxiliary space complexity. Therefore, the debugging successfully fixes the skipped-position error without changing the fundamental behavior of the algorithm.